

Tratamento de Erros com `try...catch`

Garantindo a Robustez do seu Código JavaScript

Prof. Riad Younes

UniAmérica Centro Universitário

+ descomplica

Erros em JavaScript: O Inesperado Acontece!

- **Definição:** Situações inesperadas que interrompem o programa.
- **Tipos Comuns:** `ReferenceError`, `TypeError`, `SyntaxError`.
- **Consequências:** Programa para, má experiência, perda de dados.
- **Por que Tratar Erros?** Manter estabilidade, feedback ao usuário, depuração.

Sem `try...catch`: O Risco da Interrupção

Quando um erro não é tratado, a execução do seu código é interrompida, e as linhas seguintes simplesmente não são executadas.

```
function processarDados(dados) {
  console.log(dados.nome.toUpperCase()); // Erro se dados ou dados.nome for null/undefined
  console.log("Dados processados com sucesso!");
}
processarDados(null); // Isso vai gerar um TypeError
console.log("Fim do programa."); // Esta linha NUNCA será executada
```

Resultado: O programa para abruptamente na linha do erro!

A Solução: `try...catch` - Sintaxe Básica

```
try {
  // Código que PODE gerar um erro
} catch (error) {
  // Código para lidar com o erro
}
```

Fluxo de Execução

1. O código dentro do `try` é executado normalmente.
2. Se um erro ocorre, o `try` para e o controle vai para o `catch`.
3. Se nenhum erro ocorre, o bloco `catch` é totalmente ignorado.

Bloco `try`: Onde a Ação Acontece

Propósito: Contém o código que você deseja monitorar para possíveis erros.

Escopo: Qualquer erro gerado dentro deste bloco interrompe sua execução e aciona o `catch`.

```
try {
  // Código monitorado
  let resultado = 10 / variavelInexistente;
  console.log(resultado);
} catch (error) {
  console.error("Erro capturado!");
}
```

Dica: Coloque no `try` apenas o código que realmente pode falhar para facilitar a depuração.

Bloco `catch`: O Resgate

Propósito

Captura e trata o erro lançado no bloco `try`, evitando que a aplicação trave.

O Objeto `error`

- **name:** Tipo do erro (ex: TypeError).
- **message:** Descrição do que deu errado.
- **stack:** Rastro da pilha de chamadas.

Ações Comuns

Registrar logs, exibir mensagens amigáveis ou tentar uma alternativa (fallback).



```
const styles = /* CSS */ `
  .__pathLength-2 {
    .__container {
      padding: 2rem;
      position: relative;
    }

    .__pointer {
      position: absolute;
      left: -0.5rem;
      bottom: -0.75rem;
      height: 2rem;
```

Estruturar o tratamento de erros é fundamental para códigos complexos.

Opcional: Bloco `finally`

O código dentro do **finally** sempre será executado, independentemente de ter ocorrido um erro ou não.

Uso Comum:

- Limpeza de recursos.
- Fechamento de conexões.
- Liberação de memória.

```
try {
  // Código que pode falhar
} catch (error) {
  // Lida com o erro
} finally {
  // Código que SEMPRE executa
  console.log("Execução finalizada.");
}
```

Exemplo Completo: `try...catch...finally`

```
function dividir(a, b) {
  try {
    if (b === 0) {
      throw new Error("Divisão por zero!");
    }
    return a / b;
  } catch (error) {
    console.error(error.message);
    return null;
  } finally {
    console.log("Operação finalizada.");
  }
}
```

```
CHAMADA: DIVIDIR(10, 2)
5
Operação finalizada.
```

```
CHAMADA: DIVIDIR(10, 0)
Erro: Divisão por zero!
null
Operação finalizada.
```

** Note que o **finally** executa em ambos os casos, garantindo o encerramento da lógica.*



Boas Práticas no Tratamento de Erros

Não Silencie Erros

Nunca deixe o bloco `catch` vazio. Sempre registre o erro ou tome uma ação corretiva para não perder o rastro do problema.

Seja Específico

Envolva no bloco `try` apenas o código que realmente pode falhar. Isso torna seu código mais limpo e fácil de depurar.

Feedback Amigável

Em aplicações visuais, mostre mensagens claras ao usuário final em vez de exibir erros técnicos complexos do console.

Use Logging

Utilize `console.error` ou ferramentas de monitoramento para registrar falhas em produção e facilitar a manutenção.

Atividade Individual

Validando Entradas!

Crie uma função `calcularMedia(nota1, nota2)` que trate possíveis erros de entrada.

- Use o bloco `try...catch`.
- Garanta que as notas estejam entre 0 e 10.
- No `catch`, exiba o erro no console.
- Valide se as notas são números.
- Lance um `Error` se a validação falhar.
- Retorne `null` em caso de falha.

> Dica: Use typeof nota !== 'number' e throw new Error("mensagem").

Atividade em Equipe: Processamento de Dados Externos

Desafio em Grupo

- Criem a função `carregarUsuarios()`.
- Simulem um erro (ex: 50% de chance de falha).
- Usem `try...catch` para lidar com a falha.
- Se houver erro, retornem um array vazio `[]`.
- Usem `finally` para logar o fim da tentativa.

Dúvidas e Próximos Passos



Alguma dúvida sobre `try...catch`?

O Caminho à Frente

- Tratamento de erros assíncronos (async/await).
- Criação de erros personalizados.
- Ferramentas de monitoramento (Sentry, Amplitude, New Relic).
- MDN Web Docs: `try...catch`.